Jodie Butterworth

CPT_S 440 Spring 2026

Pokemon Red Reinforcement Learning Report

AI game playing agents and their difficulties with Pokemon:

AI has well-documented problems playing the popular children's game Pokemon. This is partially based on the sparse and highly varied rewards in game progression, the number of different skills that need to be acquired, and the game's reliance on random rewards and punishments in combat and in exploring. Exploration is often difficult as the rewards are randomly scattered across the maps. Both short-term problem solving for traversing routes and winning fights is necessary as is long-term planning for collecting gym badges and defeating the elite four and champion, after finding them at the end of a large cave maze with boulder puzzles.

This RL reshaping project is one of many AI agent programs made to take on Pokemon.

Reinforcement Learning:

PPO reinforcement learning is about giving the AI a task and then training it to complete this task by repetitively running the program with rewards and punishments for different actions and outcomes. The AI learns from these rewards and punishments the actions that are desired to complete the task correctly. By favoring the sessions and actions with high rewards, the AI records what actions it should repeat. In this way, the AI programs itself based on the scores from the training.

The Pokemon Red agent uses the Gynasium API, which according to ChatGPT has the following RL space and action state processing:

At each timestep t, the environment provides:

$$s_t \rightarrow a_t \rightarrow r_t \rightarrow s_{t+1}$$

Where:

- s_t= current state observation
- a_t= action selected by the policy
- r_t= scalar reward returned by the environment

- s_{t+1}= next observed state

The objective of the PPO agent is to maximize the expected cumulative discounted reward:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where:

- $\pi$is the learned policy
- $\gamma$is the discount factor
- $T$is the episode length

The state $s_t$is a hybrid observation composed of:

1. Downsampled game screen frames
2. Temporal frame history
3. Exploration memory channels
4. Recent reward memory

ChatGPT describes the environment as using frame stacking (3 frames), to make the environment observable, and the memory uses synthetic memory channels to feed visual information into the observation tensor. For some of the rewards, such as the direction reward, the (x, y) values are pulled and measured. Other parts of the RL, like exploration, use visual data and approximate nearest neighbors. Since this reward system is Whidden's original code with some modifications with my additional rewards and penalties added to it, the algorithm is fairly complex with multiple types of rewards being summed together at different weights in the system.

The rewards are hierarchical to create an easier path of discovery or rewards for the AI.

The final reward is:

$$extra\_reward = ($$
$$r_{item} + r_{badge} + r_{area} + r\_menu + r\_combat + r\_hm +$$
$$r\_speed + r\_pokemon + r\_map + self.last\_heal\_reward)$$

$$new\_reward = ($$

$$base\_reward + extra\_reward + direction\_reward + direction\_penalty + self.last\_death\_penalty)$$

$$final_{reward} = float\big(np.clip(new_{reward}, -50, 50)\big)$$

The action space corresponds to the original Gameboy inputs, up, left, right, down, A, B. Start and Select should be disabled in this version of the code because it repetitively crashed the program during development.

Since this program uses a PPO, constraints are added to the learning to prevent destruction of the slowly acquired knowledge. The PPO algorithm can be represented as:

$$L^{CLIP}(\theta) = \mathbb{E}\big[\min\big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\big]$$

Where:

- $r_t(\theta)$= probability ratio between old and new policies
- $\hat{A}_t$= estimated advantage
- ϵ= clipping parameter

This makes PPO training slow but stable.

Originally, I was going to add heuristics to make the Whidden program (GitHub at https://github.com/PWhiddy/PokemonRedExperiments) behave more like a speedrun. I was going to add (x, y) coordinates to mimic the known speedrun path. I chose to use reward shaping to modify the reinforcement learning instead. I want the AI to teach itself to play the game rather than following my hard-coded instructions. I kept my original goals of moving quickly through the game and completing Mt. Moon, but I changed the methods so that the AI would find its own solution.

Project Goals:

- To install and run the Pokemon Red reinforcement learning program.
- To build upon the existing reinforcement training and improve the success rate of completing Mt. Moon in the original program format.
- Originally, more heuristic data was going to be used. I decided to do more reshaping of the rewards and penalties than heuristics as the project progressed.

Project Methods:

- Data on speed run times was collected from a YouTube speedrun video, @pokeguy's video: "Pokemon Red Any% Glitchless Speedrun in 1:44:03 [Former World Record]" at https://www.youtube.com/watch?v=MSOZzdIlN4A. This data was used to create a tiered reward system for getting to certain maps, like Route 1 and Viridian City in close to speedrun times, and a lower reward for double the speedrun time. This is still part of the reward system, so it is really still re-shaping.
- Penalties were added for remaining in the menu and staying in one place for too long, and for remaining on the same map for too long. These are hard values set high to not punish NPC dialogue.
- A movement penalty was added for jittering, so that the AI would not switch back and forth between buttons to simulate movement but go nowhere.
- The reward system for distance in the same direction has been repaired from the last version. It now has a tiered reward system based on the absolute value displacement of x and the absolute value displacement of y. This is sampled every second and rewarded for 12, 25 and 50. This has dramatically improved the player agent progression.
- The item reward is only for free items. If it applied to the shop, the AI would immediately spend all its money on something random. This is also because the AI must pick up the Fossil item to be able to leave Mt. Moon.
- The HM and badge rewards are to encourage game progression. Sadly, the AI never made it that far, so they are yet untested.
- The combat rewards are for high-damage hits, enemy hp reduction and enemy pokemon KOs. In mid-development, these rewards were set too high. The AI farmed the grass patch above the starting town for points and made no further progress. They have since been nerfed.
- The original healing reward could be farmed for points by getting injured and then going to heal in a loop. Healing rewards are tiered and based on the amount of HP recovery. An additional check to eliminate the automatic heal from the player fainting has been added.
- The exploration reward was nerfed from the original value for fear that it would interfere with the other rewards. Progression through the game came to a grinding halt. It has been increased until it is the single greatest reward. The reward is for each step on new (x,y) territory.
- A reward was added for adding pokemon to the party. This is not strictly necessary for game progression, but the AI wanders around and battles and faints often in Mt. Moon. Therefore, a larger party would aid in game progression through the cave in particular.
- The death penalty was in the original code, but for periods in Whidden's training, it was deactivated. It may need to be reduced in the future if it prevents early exploration.

ChatGPT describes the hierarchical nature of the rewards by separating it into long-term, mid-term and short-term goals for completing the game:

Hierarchical Interaction Between Reward Systems

Your environment combines:

Global Objectives

- exploration
- game completion
- badge acquisition

Mid-Level Objectives

- area transitions
- map progression
- HM unlocks

Local Behavioral Objectives

- smooth movement
- combat efficiency
- anti-jitter stabilization

This creates a hierarchical reward structure where:

- low-level rewards shape movement behavior
- mid-level rewards shape navigation
- high-level rewards shape long-term progression

As mentioned earlier in the combat and menu sections, there were multiple emergent behaviors that needed repair. The behavior was observed by reading the log files and observing the agent gameplay. Corrections were iteratively adjusting the rewards and penalties and troubleshooting and rewriting malfunctioning code when necessary.

ChatGPT suggested the table:

Behavior	Cause	Correction
grass grinding	combat reward too high	reduce combat rewards
healing loops	healing reward exploit	reduce healing rewards and stop healing after fainting
menu idling	lack of inactivity penalty	add penalties for staying in the same location or map for too long
directional jitter	no movement smoothness reward	add penalty for rapid directional changes with no progress and add directional progress reward

Project Results:

Sadly, I did not get the completion of Mt. Moon that I wanted. The rewards and penalties that I added often did not work, broke other existing code, or over-shadowed exploration which is essential to completing the game.

Based on the current training read-outs, the current reinforcement learning has a good balance of exploration reward, jitter penalty, menu and map penalties, distance reward, healing reward, catching pokemon reward and combat reward.

This kind of PPO reward training is time-consuming, especially on the old, more inefficient settings that work on my temperamental old computer. That said, I am not using it for anything else, so I want to continue to experiment with the reward shaping. I will leave it running for days after turning this in. Sadly, the internet breaks sometimes on the hosting computer when training is running so a restart will be required to remove the current files for submission.

Link to Powerpoint: https://github.com/WSU-JB5757/440_AI_Project/blob/main/440_final_presentation.pptx

Link to GitHub: https://github.com/WSU-JB5757/440_AI_Project.git

Note: ChatGPT generative AI was used in this project to generate code, provide installation instructions, add to this report and troubleshoot issues.